

Name: Kassim Musa Abass
Reg: SPH32/3338/2025
Program: Embedded Systems Intro

TinkerCAD: Button-Controlled LED

Objective:

To control LEDs using inputs (buttons), understand pull-up and pull-down resistors, and implement conditional logic and delays in C++.

Introduction:

This assignment explores embedding logic in a microcontroller to interact with hardware. Using an Arduino Uno wired to an LED and a push button in TinkerCAD, we implement a system that blinks an LED continuously while the button is pressed, and turns it off when the button is released.

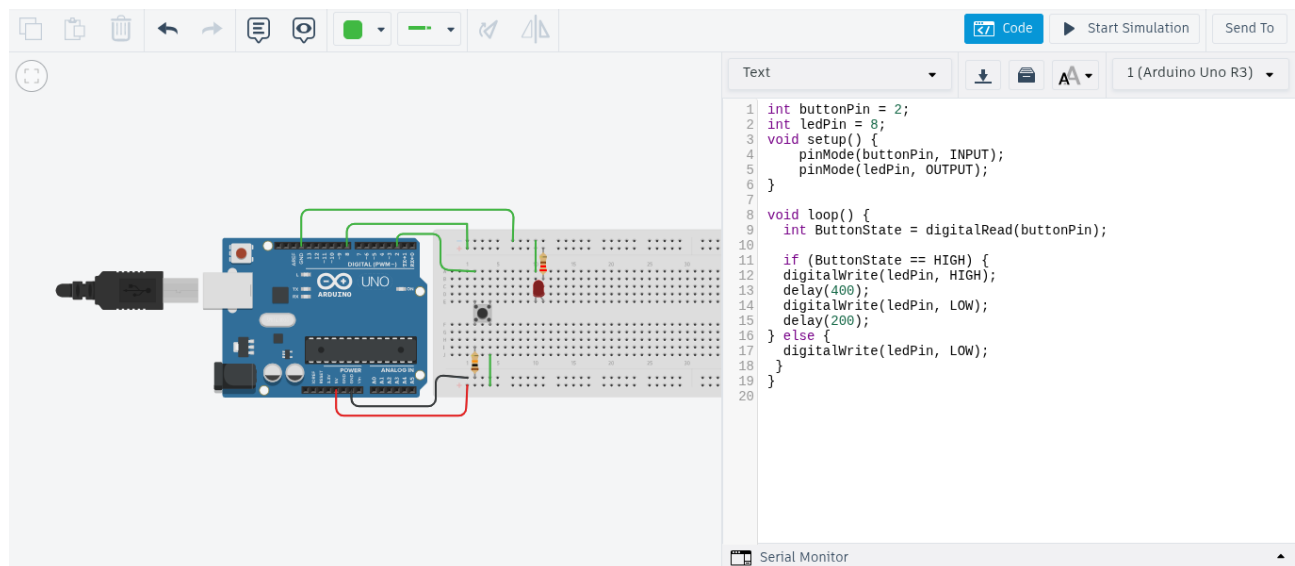
1. Theoretical Concepts:

How the Button Works: The button closes the circuit between the 5V power source and Pin 2. When pressed, the pin reads HIGH. When released, it goes LOW.

Why a Resistor is Needed: A 10k ohm pull-down resistor ensures the Arduino reading defaults cleanly to LOW instead of unpredictably 'floating'. Additionally, a 220-330 ohm resistor is necessary before the LED on Pin 8 to limit current and prevent drawing too much power, which would burn out the LED.

2. Source Code

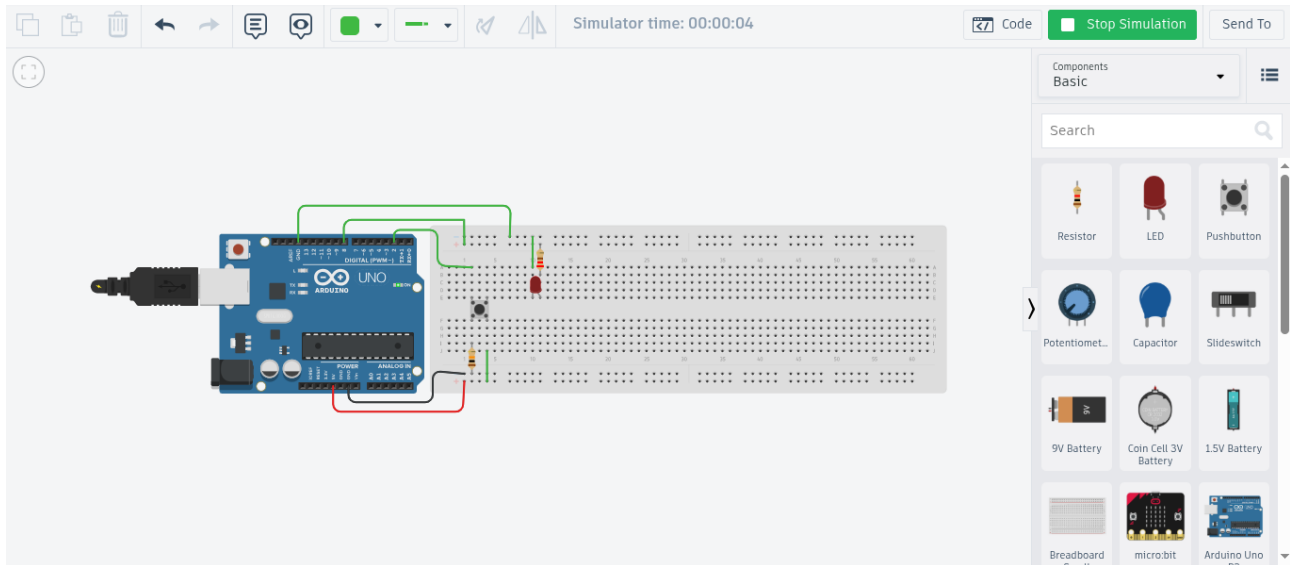
Source Code (assignment3.ino):



The screenshot shows the TinkerCAD environment. On the left, an Arduino Uno is connected to a breadboard. A push button is connected to the 5V pin of the Arduino and pin 2. A 10k ohm pull-down resistor is connected between pin 2 and ground. An LED is connected to pin 8 and ground, with a 220-330 ohm resistor in series. The code editor on the right contains the following C++ code:

```
1 int buttonPin = 2;
2 int ledPin = 8;
3 void setup() {
4   pinMode(buttonPin, INPUT);
5   pinMode(ledPin, OUTPUT);
6 }
7
8 void loop() {
9   int ButtonState = digitalRead(buttonPin);
10
11   if (ButtonState == HIGH) {
12     digitalWrite(ledPin, HIGH);
13     delay(400);
14     digitalWrite(ledPin, LOW);
15     delay(200);
16   } else {
17     digitalWrite(ledPin, LOW);
18   }
19 }
20
```

3. Circuit Diagram



4. Reflection and Challenges

What I found challenging: Implementing the flashing behavior required using delays, which stops all other execution in the program. Understanding how the `loop()` continuously checks the button state and then executes the delay-based blinking sequence when pressed was a key learning moment.